

OPENING STATEMENT

This report examines the professional perspective on integrating Docker and containerization tools into the Computer Science curriculum at the University of North Texas (UNT). The information was gathered through a structured interview with a senior software engineering professional who works extensively with containerization in industry settings. The interview surfaced three key categories of insight: a pronounced gap between industry expectations and what CS graduates currently know, specific onboarding productivity losses tied to that gap, and concrete, feasible recommendations for weaving containerization into existing UNT coursework without adding credit hours or increasing student costs.

BACKGROUND AND METHODOLOGY

The 30-minute interview with Jay Findoliya, a Senior Software Engineer at TechVista Solutions, was conducted on February 8, 2026, via Zoom. Jay brings over five years of hands-on experience deploying containerized applications in production environments and regularly mentors junior developers on DevOps practices, making him an especially credible voice on the skills gap affecting new graduates. The interview followed ten prepared questions covering Docker's role in daily development workflows, what technical knowledge companies expect from entry-level engineers, the real costs of onboarding developers unfamiliar with containerization, and practical suggestions for introducing these tools in an academic setting. With Jay's permission, the interview was recorded and transcribed for reference. Additional context was drawn from Docker's official documentation, published research on containerization pedagogy, and UNT's current course catalog. One limitation worth noting is that the findings reflect a single professional's perspective; a broader survey of employers would strengthen the generalizability of any recommendations.

FINDINGS

Industry Expectations Far Exceed Current CS Curriculum

The most striking theme from the interview was the considerable distance between what CS programs currently teach and what employers actually expect. Jay explained that containerization is no longer considered an advanced or optional skill at his company—it is a baseline requirement from a new hire's first day. Specifically, he expects incoming developers to understand the distinction between images and containers, to write basic Dockerfiles, and to use docker-compose to manage multi-service applications. Yet Jay consistently finds that recent graduates have had no hands-on exposure to these tools during their academic programs. The consequence is not merely a skills gap on paper; it translates directly into friction during the critical early months of employment, when a new engineer's ability to contribute productively is already under scrutiny. This disconnect underscores that the curriculum is lagging behind industry norms in a way that measurably affects graduates' early career experiences.

Onboarding Challenges Cost Time and Productivity

Jay provided concrete data points illustrating just how costly the gap becomes in practice. He described a typical scenario in which a new graduate spent three full days simply configuring a local development environment because they did not understand dependency management and version conflicts—problems that Docker resolves in minutes. More broadly, he estimated that engineers without prior containerization experience require two to three months of hands-on

work before they can independently containerize applications and diagnose common issues. During that period, their capacity to contribute meaningfully is limited, and senior engineers must divert significant time to covering fundamentals that universities could have addressed. Jay's framing was direct: companies are effectively providing remedial technical training that should not be their responsibility. This finding makes a strong case that the costs of the curriculum gap are borne not just by students but by the employers who hire them.

Practical Integration Opportunities Already Exist Within UNT's Structure

Rather than advocating for an entirely new course—which would add credit hours and cost—Jay recommended embedding Docker into courses that already address related concepts. He identified Software Engineering as the most natural starting point, noting that Docker would immediately eliminate the “works on my machine” problem that wastes class time during group projects. Operating Systems and senior Capstone courses were also highlighted as strong candidates because they already engage with process isolation, resource management, and deployment pipelines. Jay drew an instructive parallel to Git: containerization should be taught as a standard development tool woven across multiple courses rather than siloed into a single elective. This model of repeated, contextual exposure across the curriculum aligns well with UNT's current course structure and would require no additional credit hours, addressing the financial feasibility concern directly.

CONCLUSION

This report examined a working software engineer's perspective on the absence of containerization tools from UNT's CS curriculum. The interview revealed three interconnected findings: industry now treats Docker proficiency as a baseline expectation rather than an advanced skill; the resulting gap imposes measurable onboarding costs on both new graduates and the companies that hire them; and UNT already has courses—Software Engineering, Operating Systems, and senior Capstone projects—where Docker could be integrated without adding new courses or credit hours. These findings meaningfully strengthen the proposal for curricular reform. Jay's experience provides exactly the kind of industry evidence needed to make the case to the CS Department Chair: the gap is real, the costs are quantifiable, and the solution is practical. The most important follow-up question the proposal must still address is whether UNT faculty have the training and resources needed to teach containerization effectively, or whether professional development support would need to accompany any policy change.

REFERENCES

Findoliya, Jay. (February 8, 2026). Personal interview via Zoom.